

The Revocation Protocol

Consent Withdrawal as a Propagation Guarantee

P-RICK RESEARCH PROGRAM

an independent working-paper series on missing software

working paper P-017 v1.0 • series V

October 7, 2026 • cc by 4.0

Abstract — Every privacy interface makes the same promise — “you can withdraw anytime” — and no software system on earth can keep it. GDPR Article 7(3) elevated the promise to law in 2016: withdrawing consent must be as easy as giving it. A decade later, withdrawal is a local event with no propagation semantics: revoking an app’s OAuth token stops future API calls but says nothing about data already exported, resold, joined, or folded into a model; unsubscribing from a mailing list halts the emails while the profile that earned them keeps working; Apple’s App Tracking Transparency refuses the identifier at the device boundary while the identifier’s downstream copies persist in whatever broker graph already ingested it. This paper specifies the missing system: revocation as a protocol, not a button. We model consent as a signed, versioned grant object forming a derivation graph; define withdrawal as a graph-wide propagation event with delivery semantics (at-least-once, idempotent, acknowledgment-gated); introduce the *effective revocation time* and the *revocation gap* as measurable quantities; and specify receipts, re-attestation cycles, staleness ladders, and honeypot-based shadow audits that make non-compliance observable rather than invisible. The specification borrows its deepest lesson from certificate revocation’s failed decade — propagating bad news is economically hard, so propagate good news with short half-lives and let withdrawal shorten the schedule. The gap survives because the party that holds the data profits from it and the revoker cannot see downstream; the protocol survives because it makes silence itself evidence. Every component exists in an adjacent world; the composition — a general, cross-domain, machine-checkable revocation fabric — does not, and we state that claim so a single counterexample can refute it. **Keywords:** consent, revocation, propagation, privacy, GDPR, OAuth, consent receipts, data deletion, protocol design

Keywords: consent, revocation, propagation, privacy, GDPR, OAuth, consent receipts, data deletion, protocol design

p-rick working paper P-017 · series V (promises) · draft 1.0

Every privacy interface makes the same promise — “you can withdraw anytime” — and no software system on earth can keep it. GDPR Article 7(3) elevated the promise to law in 2016: withdrawing consent must be as easy as giving it. A decade later, withdrawal is a local event with no propagation semantics: revoking an app’s OAuth token stops future API calls but says nothing about data already exported, resold, joined, or folded into a model; unsubscribing from a mailing list halts the emails while the profile that earned them keeps working; Apple’s App Tracking Transparency refuses the identifier at the device boundary while the identifier’s downstream copies persist in whatever broker graph already ingested it. This paper specifies the missing system: revocation as a protocol, not a button. We model consent as a signed, versioned grant object forming a derivation graph; define withdrawal as a graph-wide propagation event with delivery semantics (at-least-once, idempotent, acknowledgment-gated); introduce the *effective revocation time* and the *revocation gap* as measurable quantities; and specify receipts, re-attestation cycles, staleness ladders, and honeypot-based shadow audits that make non-compliance observable rather than invisible. The specification borrows its deepest lesson from certificate revocation’s failed decade — propagating bad news is economically hard, so propagate good news with short half-lives and let withdrawal shorten the schedule. The gap survives because the party that holds the data profits from it and the revoker cannot see downstream; the protocol survives because it makes silence itself evidence. Every component

The Revocation Protocol

Consent Withdrawal as a Propagation Guarantee

P-RICK RESEARCH PROGRAM

an independent working-paper series on missing software

working paper P-017 · v1.0 · series V

October 9, 2026 · cc by 4.0

Abstract — Every privacy interface makes the same promise — “you can withdraw anytime” — and no software system on earth can keep it. GDPR Article 7(3) elevated the promise to law in 2016: withdrawing consent must be as easy as giving it. A decade later, withdrawal is a local event with no propagation semantics: revoking an app’s OAuth token stops future API calls but says nothing about data already exported, resold, joined, or folded into a model; unsubscribing from a mailing list halts the emails while the profile that earned them keeps working; Apple’s App Tracking Transparency refuses the identifier at the device boundary while the identifier’s downstream copies persist in whatever broker graph already ingested it. This paper specifies the missing system: revocation as a protocol, not a button. We model consent as a signed, versioned grant object forming a derivation graph; define withdrawal as a graph-wide propagation event with delivery semantics (at-least-once, idempotent, acknowledgment-gated); introduce the *effective revocation time* and the *revocation gap* as measurable quantities; and specify receipts, re-attestation cycles, staleness ladders, and honeypot-based shadow audits that make non-compliance observable rather than invisible. The specification borrows its deepest lesson from certificate revocation’s failed decade — propagating bad news is economically hard, so propagate good news with short half-lives and let withdrawal shorten the schedule. The gap survives because the party that holds the data profits from it and the revoker cannot see downstream; the protocol survives because it makes silence itself evidence. Every component exists in an adjacent world; the composition — a general, cross-domain, machine-checkable revocation fabric — does not, and we state that claim so a single counterexample can refute it. **Keywords:** consent, revocation, propagation, privacy, GDPR, OAuth, consent receipts, data deletion, protocol design

Keywords: consent, revocation, propagation, privacy, GDPR, OAuth, consent receipts, data deletion, protocol design

Every privacy interface makes the same promise — “you can withdraw anytime” — and no software system on earth can keep it. GDPR Article 7(3) elevated the promise to law in 2016: withdrawing consent must be as easy as giving it. A decade later, withdrawal is a local event with no propagation semantics: revoking an app’s OAuth token stops future API calls but says nothing about data already exported, resold, joined, or folded into a model; unsubscribing from a mailing list halts the emails while the profile that earned them keeps working; Apple’s App Tracking Transparency refuses the identifier at the device boundary while the identifier’s downstream copies persist in whatever broker graph already ingested it. This paper specifies the missing system: revocation as a protocol, not a button. We model consent as a signed, versioned grant object forming a derivation graph; define withdrawal as a graph-wide propagation event with delivery semantics (at-least-once, idempotent, acknowledgment-gated); introduce the *effective revocation time* and the *revocation gap* as measurable quantities; and specify receipts, re-attestation cycles, staleness ladders, and honeypot-based shadow audits that make non-compliance observable rather than invisible. The specification borrows its deepest lesson from certificate revocation’s failed decade — propagating bad news is economically hard, so propagate good news with short half-lives and let withdrawal shorten the schedule. The gap survives because the party that holds the data profits from it and the revoker cannot see downstream; the protocol survives because it makes silence itself evidence. Every component exists in an adjacent world; the composition — a general, cross-domain, machine-checkable revocation fabric — does not, and we state that claim so a single counterexample can refute it.

1 Introduction

Consider three ordinary events. A user opens her phone’s privacy settings and revokes a fitness app’s access to her health data; the app’s OAuth token dies on the spot, and three hundred kilometers away a partner analytics service that ingested her nightly heart-rate stream a year ago continues to hold, query, and enrich it. A man clicks “unsubscribe” on a retailer’s mailing list; the emails stop within days, while the behavioral profile built from two years of opens and clicks keeps feeding the retailer’s lookalike modeling, because the unsubscribe event terminated a *delivery* relationship, not a *processing* one. A regulator-mandated tracking opt-out fires on a device; the identifier refuses itself at the boundary, and every downstream copy of that identifier — already joined, hashed, synced, sold — receives no notification at all, because the opt-out was a gate, not a message.

Each event is a local success and a global failure. The interface said “withdraw anytime,” and the interface was truthful in the narrowest possible sense: the withdrawal *was* accepted, by the first party, instantly. What no system in any of these scenarios possesses is a model of what the withdrawal must *reach* — the derivation graph of everything the consent ever authorized, transitively, across organizational boundaries, including copies already materialized. Withdrawal is treated as an event that ends at the first hop; the actual object it should be terminating is a distributed data structure that nobody has an interface for.

The problem is not legal. GDPR Article 7(3) (2016) requires withdrawal to be as easy as consent was given; Article 17 grants erasure; CCPA/CPRA (2018/2020) grants deletion with a 45-day clock. The problem is that law describes *outcomes* and provides no *protocol*: no wire format for the withdrawal event, no semantics for what it covers, no delivery guarantees, no acknowledgment, no receipt, no way for a supervisory authority — or the user — to distinguish “deleted” from “ignored.” The result is an observability vacuum: compliance is asserted, violations are invisible, and enforcement is complaint-driven after invisible violations compound for years. Regulation wrote the promise; nothing implemented the keeping of it.

This paper specifies the missing system. Section 2 builds the formal model: consent as a versioned grant object, derivation as a graph relation, withdrawal as a propagation event with definable completeness, and two timestamps — *requested* revocation time and *effective* revocation time — whose difference is the measurable broken promise. Section 3 surveys the adjacent systems that each solve one slice (OAuth token revocation, Kantara consent receipts, India’s account-aggregator consent artifacts, certificate revocation’s hard-won lessons) and verifies the gap. Section 4 explains why the gap survived forty years of privacy engineering. Section 5 specifies the protocol itself: grant objects, derivation rules, revocation events, receipts, re-attestation cycles, staleness ladders, shadow audits, and the liability ladder that turns silence into evidence. Sections 6–10 give the evaluation design, the boundary with adjacent work, the objections, the limitations, and the conclusion.

The series context: this is the first paper of series V — *promises: the guarantees software implies and never has to honor*. Every interface that collects consent implies a withdrawal guarantee. This paper is that guarantee, specified.

2 A theory of revocation

2.1 Consent as a grant object

Formalize a **grant** g as a signed, versioned record:

```
g = { id, principal, processor, scope, purpose, lawful_basis, issued_at,
      expires_at, ttl, version, parent, sig }
```

The fields that do the work: **scope** is a machine-readable set of data categories (not a prose privacy policy); **purpose** binds processing to a declared class, following the purpose-limitation tradition; **ttl** is a maximum half-life after which the grant must be re-earned rather than remembered (Section 5.3 develops why TTLs, not revocation propagation, carry the load); **parent** is a back-pointer to the grant from which this one derives, making the consent graph a DAG rather than a set; **sig** makes the grant non-repudiable by its issuing principal, and versioning makes changes diffable — the property that turns a privacy policy from a document into a dataset (a move this program makes repeatedly: series IV’s defaults ledger applied it to configuration, and the same discipline applies here).

Grants *derive*: processor P1, holding grant g from user U, shares a subset of scope with processor P2 for a sub-purpose. Under the specification, P2’s possession is lawful only via a child grant $g2$ with $\text{parent} = g$, $\text{scope}(g2) \subseteq \text{scope}(g)$, and $\text{ttl}(g2) \leq \text{ttl}(g)$. Derivation without a child grant is exactly what the law calls onward transfer without basis — today it is invisible, and under the protocol it is *structurally* unrecordable in compliant form, which is the point of formalizing at all.

2.2 The revocation gap

When U withdraws consent at time t_R for grant g , define:

- **Requested revocation time** $\text{RRT}(g) = t_R$: the moment the principal acts.
- **Effective revocation time** $\text{ERT}(e)$ for each edge e derivable from g : the moment processing on e actually ceases.
- **The revocation gap** $\Delta(e) = \text{ERT}(e) - \text{RRT}(e) \geq 0$.
- **Revocation completeness** $\kappa(g) = |\{e : \Delta(e) < \Delta_{\max}\}| / |\text{edges}(e \text{ derivable from } g)|$, for a regulatory bound $\Delta_{\max}$.

These three quantities make the promise falsifiable. “Withdraw anytime” currently has no defined semantics beyond the first hop; with them, a system either meets $\kappa = 1$ within $\Delta_{\max}$ or it does not, and the difference is measured, not litigated. The analogy is deliberately operational: TCP does not define reliability as a vibe, it defines it as a sequence of acknowledgments and timeouts; the same discipline is applied here to a promise that is presently enforced by complaint letters.

The subtlety the definitions expose: ERT is defined *per edge*, because revocation is not one event but a family. The mailing list unsubscribe achieved ERT on the delivery edge (emails stopped) and never even attempted ERT on the processing edges (profiling, modeling). The OAuth token revocation achieved ERT on the API-call edge and was silent on the ingest edge. Today’s systems each terminate exactly the edge their own product controls — which is to say, they are not failing at revocation; they are succeeding at a much smaller thing that wears revocation’s name.

2.3 Withdrawal, deletion, forgetting

The protocol distinguishes three distinct operations that current UI language collapses into “withdraw”:

1. **Cessation** — stop new processing. Cheapest; locally achievable; what every current button actually does.
2. **Deletion** — destroy materialized copies and derived records. Requires delivery semantics downstream; provable only via receipt and audit, not physics (Section 7).
3. **Forgetting** — remove the *influence* of the data from models, aggregates, and indexes. Partially achievable through retraining triggers and contribution-tracking; in general unachievable exactly, and the honest position is to bound it, not promise it.

The specification machinery (events, receipts, audits) is shared across the three; the guarantees differ deliberately. Conflating them is how interfaces make promises that are simultaneously trivial (cessation) and impossible (perfect forgetting); separating them is how an implementable guarantee reappears.

3 The landscape: every slice solved, no composition

The claim of this section is falsifiable: *no deployed system implements cross-processor, receipt-carrying, version-aware revocation propagation for general consent*. Each adjacent system below implements a fragment; the table at the end grades them.

OAuth 2.0 token revocation (RFC 7009, 2013). The closest everyday object. A client POSTs a token to a revocation endpoint; the authorization server invalidates that token, and RFC 7009 §2 notes the *possibility* of cascading to related tokens. But the revocation universe ends at the authorization server’s own token table: it is a protocol for killing *credentials*, not *consequences*. Data already delivered through the token is out of scope by design. Verdict: solves cessation at exactly one hop, one issuer, no derivations, no receipts to third parties.

Kantara Consent Receipts (2017). A receipt format issued *at consent time*: what was collected, for what purpose, with what policy. Its own specification names the follow-on work — an “updated consent receipt” to record changes — and the ecosystem has not built the update layer at scale. Consent receipts record the *birth* of consent with no machinery for its death. Verdict: the issue-time half of the problem, elegant and stalled.

UMA 2.0 (User-Managed Access, Kantara). The closest live protocol in spirit: a resource owner delegates access via claims-scoped tokens that expire, and monitors an introspection endpoint. But UMA governs *live access* to a resource server’s guarded resources; once data leaves the guardian, UMA’s model ends. It is a door with excellent locks and no memory of what walked out. Verdict: strong single-resource delegation, no derivation graph, no propagation.

India’s account aggregator / DEPA consent artifacts (2020–). The strongest existence proof that the machinery works at national scale: purpose-bound, TTL-carrying, revocable data-sharing artifacts, regulator-audited, with escrowed revocation semantics. The caution for this paper’s claim is honest: revocation *is* propagated across the AA fabric. But the domain is financial data inside a licensed, closed, four-party architecture — the protocol is mandatory within a regulated perimeter and does not exist as a general fabric for the open ecosystem of ordinary consent (a fitness app, a retailer, a school form). Verdict: proves the mechanics; confined to one regulated domain; not general. This is recorded as the strongest adjacent system and the reason the paper’s claim is scoped to *general, cross-domain* fabrics.

GDPR/CCPA/CPRA. The promise’s legal layer: withdrawal as easy as giving (Art. 7(3)), erasure (Art. 17), deletion with a 45-day clock (CCPA §1798.105). Mandates outcomes; specifies no wire format, no receipt, no registry; enforcement is complaint-driven. The supervisory authorities receive individual complaints the way mail rooms receive letters. Verdict: the promise, without a protocol to keep it.

Table 1: Table 1: the landscape, graded.

System	Cessation	Deletion	Forgetting	Derivations	Receipts	Cross-org	General domain
OAuth RFC 7009	STRONG	none	none	none	no	no	credentials
Kantara receipts	none	none	none	none	birth-only	no	issue-time
UMA 2.0	STRONG	none	none	single	partial	no	guarded resources
DEPA/AA fabric	STRONG	PARTIAL	none	registry-bound	yes	regulated 4-party	financial data
GDPR/CCPA	legal only	legal only	legal only	undefined	none	by complaint	all
GPC signal	one bit	none	none	none	no	broadcast	opt-out
Apple ATT	device-gate	none	none	none	no	no	tracking ID
Platform deletion callbacks	PARTIAL	one hop	none	none	platform-side	one ecosystem	app data
Certificate short-TTLs	STRONG	n/a	n/a	none	n/a	yes	certificates
This spec	STRONG	receipt-bound	bounded	DAG	signed, chained	yes	general consent

Global Privacy Control (GPC). A browser signal expressing a legal opt-out. A broadcast, not a conversation: no per-processor state, no acknowledgment, no downstream coverage. Verdict: one bit, one direction, no receipts.

Apple App Tracking Transparency (2021). Refuses the identifier *at the device boundary* — a gate, not a message. Post-ATT research documented the partial nature of the boundary and the adaptability of fingerprinting; the already-sold copies of identifiers were never in ATT’s model. Verdict: excellent cessation gate; zero propagation.

Platform data-deletion endpoints. Meta and Google require apps to expose deletion callbacks as a platform policy. Real, enforced, and single-platform: the callback reaches the app, not the app’s own downstream; the platform audits its own perimeter. Verdict: deletion propagation one hop deep, inside one company’s app ecosystem, by contract.

Certificate revocation: CRL, OCSP, and the short-lived-cert lesson. The deepest prior art, and the source of this paper’s most important design decision. The web spent a decade trying to propagate revocation of bad certificates and largely failed on economics: CRLs were too big, OCSP added latency and was soft-failed by browsers that preferred availability to rigor, and the ecosystem converged on the solution that *stopped propagating bad news* — short-lived certificates (Let’s Encrypt’s 90-day and shorter lifetimes) that make revocation mostly unnecessary by making trust re-earned frequently. The lesson imported wholesale: **do not build a system whose correctness depends on bad news traveling; build one where good news travels on a short schedule and withdrawal collapses the schedule.** Consent re-attestation cycles (Section 5.4) are the direct application.

Adjacent evidence, no interface: takedown and counternotice. DMCA-style takedown is the one revocation-like fabric that demonstrably propagates across independent parties at scale (search engines, hosts, registries) — with notices, counter-notices, and repeat-infringer consequences. It is content-specific, legally forceful, and adversarial by design; the general consent case lacks its lever, which is precisely the gap the liability ladder (Section 5.7) is designed to supply.

The verdict in this program’s grading language: components PARTIAL-to-STRONG (every mechanism exists somewhere), composition STRONG-vacant (no system composes them for general consent).

The one near-miss — DEPA — is domain-bound and architecturally closed; generality is not a deployment detail but a different object.

4 Why the gap survived

The holder profits from the residue. Every byte downstream of a withdrawal is an asset to the holder and a liability only in the event of an audit that cannot see it. The incentive gradient points exactly at the gap. This is not a moral claim but a structural one; protocol design that assumes volunteerism at the point where incentives reverse is design that will not deploy. The specification’s answer is not to reverse the incentive but to *price the silence* (Sections 5.6–5.7): make non-propagation observable enough that its expected cost changes.

The revoker cannot see downstream. The principal withdrawing consent has, at best, a first-hop list (OAuth’s granted-apps screen, the newsletter’s unsubscribe). She cannot enumerate derivations, so she cannot test compliance, so she cannot complain accurately. In security terms, the data subject lacks egress visibility — the same asymmetry that made network intrusion detection necessary. The shadow-ledger instruments of Section 5.6 are the intrusion-detection analog for personal data.

Distributed naivety. Each processor in the chain honestly believes “we honor revocations” about its own hop — the app deleted its copy; the partner deleted its copy “per contract”; the sub-processor “per policy.” No single actor is lying; the composition is. Unowned composition failure is the signature of missing *infrastructure* rather than missing intent, and it is the signature this program’s other series keep finding (the defaults ledger found it in configuration, the complexity ledger in design).

The email-era interface froze. “Unsubscribe” was invented when the consent object *was* a mailing list entry — one row, one hop, termination was deletion. The interface survived the migration of consent from rows-in-a-list to graphs-of-derived-assets, carrying with it the assumption that withdrawal ends at the first server. Every privacy UI since has been a re-skin of that 1996 shape.

No wire format, so no market. Vendors cannot sell revocation-propagation middleware because there is nothing for it to speak; regulators cannot check compliance because there is nothing to inspect; auditors cannot certify because there is nothing to sample. The absence of a protocol is the absence of an industry — which is also why building the protocol first, before the product, is the correct order for this program.

5 The specification

The protocol has seven components. Each is specified to the depth an implementer could build against; together they form the Revocation Fabric (working name; the paper is indifferent to branding).

5.1 Grant objects and the derivation DAG

Grants (Section 2.1) are issued in JSON-LD-signed envelopes (the format choice is illustrative; the requirements are signature, versioning, and parent-pointers). Issue-time is receipted in the Kantara tradition — the birth certificate — but unlike Kantara, the object carries its own liveness budget (ttl) and its derivation policy (derive: {allowed: bool, max_scope_subset: float, purpose_narrowing: required}).

Onward transfer under the protocol: P1 may only share with P2 by *co-issuing* a child grant signed by *both* P1 (as grantor-processor) and the principal U (as data owner). The double-signature requirement is the load-bearing wall: it means no lawful derivation can exist without a record whose lineage points back to U’s original grant. Silent derivation — the everyday reality of today’s ad-tech and analytics resellers — becomes structurally unrecordable in compliant systems, and *only recorded derivations are processable*: downstream processors that receive data without an accompanying child grant are, by protocol definition,

processing ungranted data, which converts a privacy question into a contract question any auditor can check mechanically.

The DAG is stored where the consent was issued — a **grant ledger** — mirrored to the principal’s personal store. This is the event-bus family reunion: series I’s personal event bus (P-001) specified the capture and durability layer; a grant is simply one more event type in that ledger, and the revocation protocol composes with it rather than replacing it.

5.2 The revocation event

```
revoke = { grant_id, versions: [all | enumerated], scope: [all | enumerated
categories], operations: [cessation, deletion, forgetting-bounded], issued_at,
sig_principal, nonce }
```

Three coverage dimensions, independently addressable: *which versions* (the versioning of grants means a withdrawal can target today’s terms without litigating yesterday’s), *which categories* (partial withdrawal — “stop the location sharing, keep the receipts” — which current UIs make impossible because consent was never category-addressable at the point of re-withdrawal), and *which operations* (cessation-only, cessation+deletion, or the full bounded package). The nonce defeats replay. The signature makes the withdrawal non-repudiable, which matters because the receipt architecture makes the *absence of response* legally meaningful — that only works if the withdrawal itself is beyond dispute.

5.3 Propagation semantics

Delivery is **at-least-once, idempotent, acknowledgment-gated**, on a bounded schedule:

- **Push window $\Delta_{\max}$** — every processor in the derivation DAG must acknowledge the revocation event within a regulatory bound (the DEPA fabric’s working analog is measured in days; the general web’s honest bound is discussed in Section 9). Idempotency keys on `grant_id + versions + nonce` make re-delivery free and replay harmless.
- **Escalation on silence** — unacknowledged processors are re-queried on a doubling backoff, and every silence is *recorded*, timestamped, and visible: to the principal, to the grant ledger, and — at thresholds — to the audit fabric (Section 5.6).
- **The staleness ladder** — a grant’s confidence decays toward its TTL; a revocation *collapses the schedule*, forcing the next re-attestation cycle to arrive immediately. The certificate lesson (Section 3) is applied structurally: the protocol does not depend on the withdrawal traveling; it depends on the *renewal* schedule, which the withdrawal accelerates. A processor that simply ignores revocation events still stops lawful processing at the next renewal it fails to obtain — the “graceful degradation” of the bad-news-propagation problem.

5.4 Re-attestation cycles

Every grant’s TTL schedules a renewal: the processor re-requests, the principal re-grants (one interaction, by the Art. 7(3) symmetry), and the renewal re-signs the derivation DAG one level at a time. Defaults matter here and are borrowed from the series IV defaults ledger: renewal is *not* automatic silence-extends-consent; silence *decays* it. A processor that wants to keep processing must ask again, the way Let’s Encrypt asks again every 90 days rather than asking once and remembering forever. The economic consequence is the inversion the certificate ecosystem discovered: consent becomes cheap to renew and expensive to assume — and the revocation button becomes the emergency brake on a train that is already stopping itself on schedule.

5.5 The receipt registry

Each processor returns a signed **revocation receipt** for each covered derivation:

```
receipt = { revoke_id, processor, derived_copies: [inventory classes],
method: [purge | crypto-shredded | third-party-held], completed_at, attestation_
cycle_next, sig_processor }
```

Receipts are tamper-evident, chained (each attestation cycle's receipt references the prior), stored in the grant ledger, and — critically — *deletable only by the principal*. The registry is the first artifact that makes “we deleted your data” an inspectable claim with a signature on it rather than an email from a support address. It does not prove deletion (nothing proves deletion; Section 7); it proves the making and maintenance of the claim, which is the strongest provable thing and, as Section 5.7 argues, the legally load-bearing one.

5.6 Shadow audits and the honeypot ledger

Observability of *non-compliance* cannot come from the compliant channel alone. The protocol's audit fabric plants **canary records** at issue time: synthetic, uniquely-marked data objects (a distinctive name, a one-off email alias, a synthetic device fingerprint) issued *only* under a specific grant. Post-revocation, any downstream appearance of a canary — a marketing email to the alias, a lookup hit in a broker's enrichment API, the canary identifier surfacing in a breach corpus — is direct, timestamped evidence that some edge in the revoked subgraph was never terminated. The technique is the privacy-audit cousin of intrusion canaries and egress traps: instead of proving the attacker got in, it proves the data got out, after it promised not to. Coverage is probabilistic and stacking — a hundred canaries planted across categories raise detection probability for silent derivations; each hit also *localizes* the leak to the edge whose grant the canary rode. This is the instrument that converts the observability asymmetry of Section 4 from a permanent condition into an audit design.

5.7 The liability ladder

Silence, at each rung, costs more than the rung below:

1. **Contractual** — processors that co-sign child grants accept receipt obligations; silence past $\Delta_{\max}$ is a per-event contract breach with liquidated damages (the DMCA lever, imported).
2. **Presumptive** — after a signed revocation event and a recorded $\Delta_{\max}$ breach, the burden of proof inverts: the processor is *presumed* to be processing without basis unless its receipt chain shows otherwise. The presumption is rebuttable, but the rebuttal requires the registry artifacts the protocol defines — which is the designed coupling: the protocol creates exactly the evidence class that the presumption demands.
3. **Procurement-grade** — receipt-coverage ratios become a procurable metric (“99.4% effective-revocation coverage, median gap 11 days, attested”) the way SOC 2 and uptime SLAs are procurable. Enterprise procurement is the fastest enforcement engine in software history; the spec is deliberately shaped so it can grip.
4. **Regulatory** — supervisory authorities receive aggregate gap statistics and canary-hit localizations, not individual letters: enforcement shifts from mail-room to dashboard, the single largest lever the protocol supplies.

5.8 The withdrawal surface

The Art. 7(3) symmetry requirement becomes a machine-checkable UI contract: *the withdrawal interaction cost must be $\leq$ the grant interaction cost* — one tap, on the same surface, in the same session context, or the grant ledger records the asymmetry as a compliance event. Making ease-of-withdrawal a *measured property of the interaction graph* — rather than an aspiration in a recital — is a small formalism with a large effect, because it converts the most-cited and least-enforced sentence in GDPR into a testable assertion.

6 Evaluation design

The protocol is evaluable on four instruments, in increasing order of cost:

1. **Revocation-latency measurement.** Implement the event format against a test federation of 20 processors across 3 derivation levels; instrument RRT and ERT per edge; report the Δ distribution and κ by $\Delta_{\max}$. Success criterion: the machinery demonstrably measures the gap — the metric is the product.
2. **Coverage audit via canaries.** In an authorized test bed, plant 100 honeypot records across 5 data categories, revoke, then monitor 12 downstream observation points (alias inboxes, enrichment APIs, ad-reset signals) for 180 days. Report detection probability per category and localization accuracy per hit.
3. **Receipt-forgery resistance.** Red-team the receipt chain: attempt backdated receipts, receipt substitution at renewal, and co-signature forgery; measure detection by the ledger’s chaining and signature verification. This is the adversarial pass this program requires of its own specifications.
4. **Longitudinal field deployment.** A single high-trust domain first (a university’s alumni-data sharing, a hospital’s research-consent fabric — domains where the principal population is captive and the processors are auditable), instrumented for one renewal cycle. The DEPA fabric’s national-scale precedent suggests the mechanics hold; the field test’s question is whether the *general* wire format survives contact with heterogeneous processors.

7 What this is not

Not a deletion oracle. Copies cannot be un-copied; no protocol proves the absence of a byte in a party’s basement. The claim is narrower and honest: the protocol makes non-compliance *observable* (canaries), non-response *presumptive* (ladder), and renewal *conditional* (TTLs). Perfect enforcement is not on offer; priced silence is.

Not forgetting. Model-internal residues (weights trained on revoked data) are bounded, not erased, and the bounding is honest: contribution-tracking triggers retraining at thresholds, and beyond that the paper claims nothing. The model-extinction paper (P-011) owns the behavioral-residue territory; this protocol supplies it the trigger events it lacks.

Not an anonymity system. Data minimization is upstream of consent; pseudonymization is orthogonal. The revocation fabric terminates *authorizations*, not identities.

Not blockchain-dependent. The ledger needs tamper-evidence, not consensus; a signed append-only log at the issuing surface suffices, and the design assumes nothing costlier than a notary-grade hash chain.

8 Objections, confronted

“Data holders will ignore it.” Outside regulated perimeters, some will — exactly as some ignored DMCA and OSCP. The design’s response is the certificate ecosystem’s lesson inverted: the TTL schedule makes ignoring *expensive by default* (processing authority expires on its own), and the ladder prices *remaining* silent. The realistic deployment thesis is not universal voluntary compliance; it is procurement pull (rung 3) making the protocol a market requirement the way SOC 2 became one.

“Users will not bother revoking.” The interaction economics answer: the protocol’s withdrawal is one tap *because the grant ledger made consent addressable*, and re-attestation cycles mean the user manages consent at renewal frequency, not in crises. The privacy-paradox literature cuts the other way here: low-cost withdrawal raises withdrawal rates, which is precisely why holders historically priced withdrawal high — and why the machine-checkable symmetry of Section 5.8 exists.

“GDPR already mandates this.” GDPR mandates the *outcome* and specifies no *mechanism*; no wire format, no receipts, no observability. A law without a protocol is a promise without a postal service — and the decade since Art. 7(3) is the natural experiment: the outcome is broadly unmet, and the unmet part is exactly the part no protocol covers.

“Propagation across the open web is impossible.” For anonymous, cash-and-carry data flows, true — and the spec does not claim that regime. Consent regimes operate in *identified B2B graphs* (app↔platform, retailer↔partner, hospital↔research sponsor), where parties are contractual, reachable, and auditable — the same graphs where DMCA propagates and platform deletion callbacks already fire. The open-web residue is the honest boundary, and it is the same boundary every enforcement architecture in this program’s prior series has accepted (the bus factor protocol governs registries, not vandals).

“Short TTLs alone suffice; skip revocation.” TTLs cover *decay*, not *directed* withdrawal: a user revoking today does not want the data to expire gracefully in eight months — and TTLs say nothing about derivations already materialized into models and lookalike tables. The two mechanisms are complements, and the revocation event is what collapses the TTL schedule on demand.

9 Limitations

The general-web residue (Section 8) is real: below the contractual waterline, the protocol’s instruments degrade to canary evidence and presumption, not enforcement. $\Delta_{\max}$ for the open ecosystem is a policy parameter this paper does not fix; honest values differ by two orders of magnitude across jurisdictions, and the spec treats it as configurable rather than pretending one number fits all. Canaries are probabilistic, and a sophisticated holder can avoid seeded records at some cost — the audit arms race is genuine, though each escalation raises the holder’s marginal cost, which is the ladder’s design intent. Cold-start is the classic protocol bootstrapping problem: the fabric is worthless at one processor and compounding at a thousand, so the deployment path runs through regulated perimeters first (the DEPA sequence, deliberately). And the model-residue bound inherits every limitation P-011 catalogued for behavioral conservation; this protocol’s contribution is the trigger, not the solvent.

Citations recorded from domain knowledge carry this program’s standing honesty mark — *verification queued* — and pass through the live-source ledger before final release; the paper’s load-bearing claims (RFC 7009’s scope, Kantara’s issue-time focus, DEPA’s revocation mechanics, ATT’s boundary behavior) are each independently checkable against primary sources.

10 Conclusion

Every consent interface on earth implies a guarantee that no system implements: that “withdraw” is a verb with an object, that it reaches what the consent touched, that stopping is distinguishable from ignoring. The law has demanded the outcome for a decade; the missing artifact is the protocol — grant objects with lineage, withdrawal events with coverage, delivery with receipts, renewal on a schedule that consent-holders cannot assume their way past, audits that canary the silence, and a liability ladder that turns absence of response into presence of evidence. The certificate ecosystem spent fifteen years and billions of failed-propagation dollars to teach the load-bearing lesson: do not propagate the bad news; schedule the good news, and let withdrawal be the schedule’s collapse. The revocation protocol is that lesson, applied to the largest class of unkept promises in software. The world does not lack consent. It lacks the keeping.

References

Verification-queued marks follow the program’s citation-honesty convention; primary sources are checked through the live-source ledger before final release.

1. European Union. *Regulation (EU) 2016/679 (GDPR)*, Articles 5, 7(3), 17. 2016.
2. Jones, M., Bradley, J., Sakimura, N. *RFC 7009: OAuth 2.0 Token Revocation*. IETF, 2013.
3. Kantara Initiative. *Consent Receipt Specification v1.1*. 2017.
4. Kantara Initiative. *User-Managed Access (UMA) 2.0*. 2018–2019.
5. Reserve Bank of India / DEPA architecture working group. *Data Empowerment and Protection Architecture: consent artifacts, purpose, and revocation*. 2020. (*verification queued*)
6. California Legislature. *CCPA §1798.105; CPRA amendments*. 2018, 2020.
7. W3C Privacy Community Group. *Global Privacy Control (GPC) specification draft*. (*verification queued*)
8. Apple Inc. *App Tracking Transparency documentation and App Store policy*. 2021.
9. Meta Platforms. *Platform data deletion endpoint requirements, developer documentation*. (*verification queued*)
10. Lindgren, D. et al. *Post-ATT measurement studies*. 2021–2023. (*verification queued*)
11. ISRG / Let’s Encrypt engineering blog line on revocation and short-lived certificates — “Announcing Six Day and IP Address Certificate Options in 2025” and the certificate-lifetime rationale posts (90→64-day default path). (*verified 2026-10; the earlier named-author attribution is withdrawn — these posts are institutional.*)
12. de la Cuadra, F. *OCSP soft-fail behavior in major browsers*. (*verification queued*)
13. United States Code. *17 U.S.C. §512 — DMCA notice-and-takedown*.
14. Rescorla, E. “Security Holes ... Who Cares?” *12th USENIX Security Symposium*, 2003 — the observational study (post-OpenSSL July 2002) that argues revocation’s failure modes are economically structural. (*verified 2026-10; title corrected — there is no USENIX paper titled “On the economics of certificate revocation.”*)
15. Acquisti, A., Brandimarte, L., Loewenstein, G. “Privacy and human behavior in the age of information.” *Science* 347. 2015.
16. Delacroix, S. *Recursive consent and downstream-data-flow scholarship*. 2021. (*verification queued*)
17. p-rick research program. *P-001 The Personal Event Bus; P-005 Provenance-Native Storage; P-011 Model Extinction; P-016 The Defaults Ledger*. 2026.
18. OpenSSL Foundation. *X.509 CRL/OCSP operational history*. (*verification queued*)